

02_Graph_Theory_Tree 树-板子

树的定义

[P11242 碧树 - 洛谷](#)

```
#include<bits/stdc++.h>
// #define int long long
using namespace std;
using ll=long long;

void work(){
    int n;
    cin>>n;
    int maxx=-1;
    vector<int> a(n);
    for(int i=0;i<n;i++){
        cin>>a[i];
        maxx=max(maxx,a[i]);
    }
    cout<<maxx+n-1;
} signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t=1;
    // cin>>t;
    while(t--){
        work();
    }
}
```

树的直径

[B4016 树的直径 - 洛谷](#)

```
#include<bits/stdc++.h>
// #define int long long
using namespace std;
using ll=long long;

void work(){
    int n;
    cin>>n;

    vector<set<int> > g(n+1);
    for(int i=1;i<n;i++){
        int x,y;
        cin>>x>>y;
        g[x].insert(y);
        g[y].insert(x);
    }
}
```

```

vector<int> dep(n+1);
auto dfs=[&](auto &&self,int x,int p) ->void{
    for(auto y:g[x]){
        if(y==p) continue;
        dep[y]=dep[x]+1;
        self(self,y,x);
    }
};

dfs(dfs,1,-1);
int u=max_element(dep.begin(),dep.end())-dep.begin();
fill(dep.begin(),dep.end(),0);

dfs(dfs,u,-1);
// int v=max_element(dep.begin(),dep.end())-dep.begin();

int d=*max_element(dep.begin(),dep.end());

cout<<d;
}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int t=1;
    // cin>>t;
    while(t--){
        work();
    }
}

```

树的重心:

[P1395 会议 - 洛谷](#)

```

#include<bits/stdc++.h>
// #define int long long
using namespace std;
using ll=long long;

void work(){
    int n;
    cin>>n;

    vector<set<int> > g(n+1);
    for(int i=1;i<n;i++){
        int x,y;
        cin>>x>>y;
        g[x].insert(y);
        g[y].insert(x);
    }
}

```

```

int cen=-1;
int ans=1e9;

vector<int> dep(n+1,0),siz(n+1,0);
auto dfs=[&](auto &&self,int x,int p) -> int{
    siz[x]=1;
    int con=0;
    for(auto y:g[x]){
        if(y==p) continue;
        dep[y]=dep[x]+1;
        int sy=self(self,y,x);
        siz[x]+=sy;
        con=max(con,sy);
    }
    con=max(con,n-siz[x]);

    if(con<ans){
        ans=con;
        cen=x;
    }
    else if(con==ans){
        cen=min(x,cen);
    }
    return siz[x];
};

dfs(dfs,1,-1);
dep.assign(n+1,0);
siz.assign(n+1,0);

dfs(dfs,cen,-1);

int ssum=0;
for(int i=1;i<=n;i++){
    ssum+=dep[i];
}
cout<<cen<<" "<<ssum<<'\\n';

}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int t=1;
    // cin>>t;
    while(t--){
        work();
    }
}
/*
*/

```

树的中心

```
#include<bits/stdc++.h>
using namespace std;
/*
题目：数的中心
n个节点，n-1条边，无边权，选择一个点使其到最远的节点的距离最小
*/
vector<vector<int>>>mat;
vector<int> d1; //i节点向下走的最长链长度
vector<int> d2; //i节点向下走的次长链长度
vector<int> up; //i节点向上走的最长链长度
int minn = 1e9; //维护min(max(d1[i],up[i]));
int n; //节点数
//找到点x使得max(d1x,upx)最小，那么x即为树的中心
int x,y;

void dfsd(int cur,int fa){ //向下走
    //求d1和d2
    for(int nxt : mat[cur]){
        if(nxt == fa) continue;
        int w = 1;
        dfsd(nxt,cur);
        if(d1[nxt] + w > d1[cur]){
            d2[cur] = d1[cur];
            d1[cur] = d1[nxt] + w;
        }else if(d1[nxt]+w>d2[cur]){
            d2[cur] = d1[nxt] + w;
        }
    }
}

void dfsu(int cur,int fa){ //通过
    for(int nxt : mat[cur]){
        if(nxt == fa) continue;
        int w = 1;
        up[nxt] = up[cur] + w;
        if(d1[nxt]+w != d1[cur]){
            up[nxt] = max(up[nxt],d1[cur]+w);
        }else{
            up[nxt] = max(up[nxt],d2[cur]+w);
        }
        dfsu(nxt,cur);
    }
}

void GetCenter(){
    dfsd(1,0);
    dfsu(1,0);
    for(int i=1;i<=n;i++){
        if(max(d1[i],up[i])<minn){
            minn = max(d1[i],up[i]);
            x = i;
        }
    }
}
```

```

        y = 0;
    }else if(max(d1[i],up[i])==minn){
        y = i;
    }
}
}

```

```

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);

    cin>>n; //节点个数
    mat.assign(n+1,vector<int>(0));
    d1.assign(n+1,0);
    d2.assign(n+1,0);
    up.assign(n+1,0);

    for(int i=1;i<=n-1;i++){
        //输入节点（无边权）
        int u,v; cin>>u>>v;
        mat[u].push_back(v);
        mat[v].push_back(u);
    }
    GetCenter();
    cout<<x<<"\n"<<y<<"\n";

}

```

```

/*
11
1 2
1 5
2 3
2 8
3 4
8 10
5 9
5 6
9 11
6 7
输出1 0（0表示只有一个中心）
*/

```

子树的大小及深度

[U261073 子树的大小及深度 - 洛谷](#)

```

#include<bits/stdc++.h>
//#define int long long
using namespace std;
using ll=long long;

```

```

void work(){
    int n;
    cin>>n;

    vector<set<int> > g(n+1);
    for(int i=1;i<n;i++){
        int x,y;
        cin>>x>>y;
        g[x].insert(y);
        g[y].insert(x);
    }

    vector<int> par(n+1,-1),siz(n+1),dep(n+1);

    auto dfs=[&](auto &&self,int x) -> void{
        siz[x]=1;
        for(auto y:g[x]){
            if(y==par[x]) continue; //父节点不算
            par[y]=x; //节点y的父节点是x
            dep[y]=dep[x]+1; //所以节点y所在深度是节点x所在深度+1
            self(self,y); //继续遍历节点y
            siz[x]+=siz[y];
        }
    };
    int root=1;
    dep[root]=1;
    dfs(dfs,root);
    for(int i=1;i<=n;i++){
        cout<<siz[i]<<" "<<dep[i]<<'\\n';
    }
}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    int t=1;
    // cin>>t;
    while(t--){
        work();
    }
}
/*
6
1 2
5 2
2 3
4 2
5 6

6 1
5 2

```

1 3

1 3

2 3

1 4

*/